🌲 DAY 2: MASTER ARROW WORKFLOWS & COMPONENT EXECUTION TREES

Traditional Machine Learning & Parallel Computing using Intel oneAPI

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 2 (Strictly 3 Hours / 180 Minutes)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

🗺 MASTER INDEX OF DAY 2 ARROW TREES

- Tree 1: Parallel Computing & Amdahl's Law Scaling Tree
- Tree 2: OpenMP Fork-Join & Shared-Memory Execution Flow
- Tree 3: The False Sharing & Cache Coherency (MESI) Invalidation Flow
- Tree 4: Distributed-Memory MPI Process Communication Tree
- Tree 5: Intel AI Analytics Toolkit Architecture & Hardware Dispatch Flow
- Tree 6: Scikit-learn Dynamic Monkey-Patching (`sklearnex` / oneDAL) Flow
- Tree 7: The Complete End-to-End Traditional ML on HPC Lifecycle

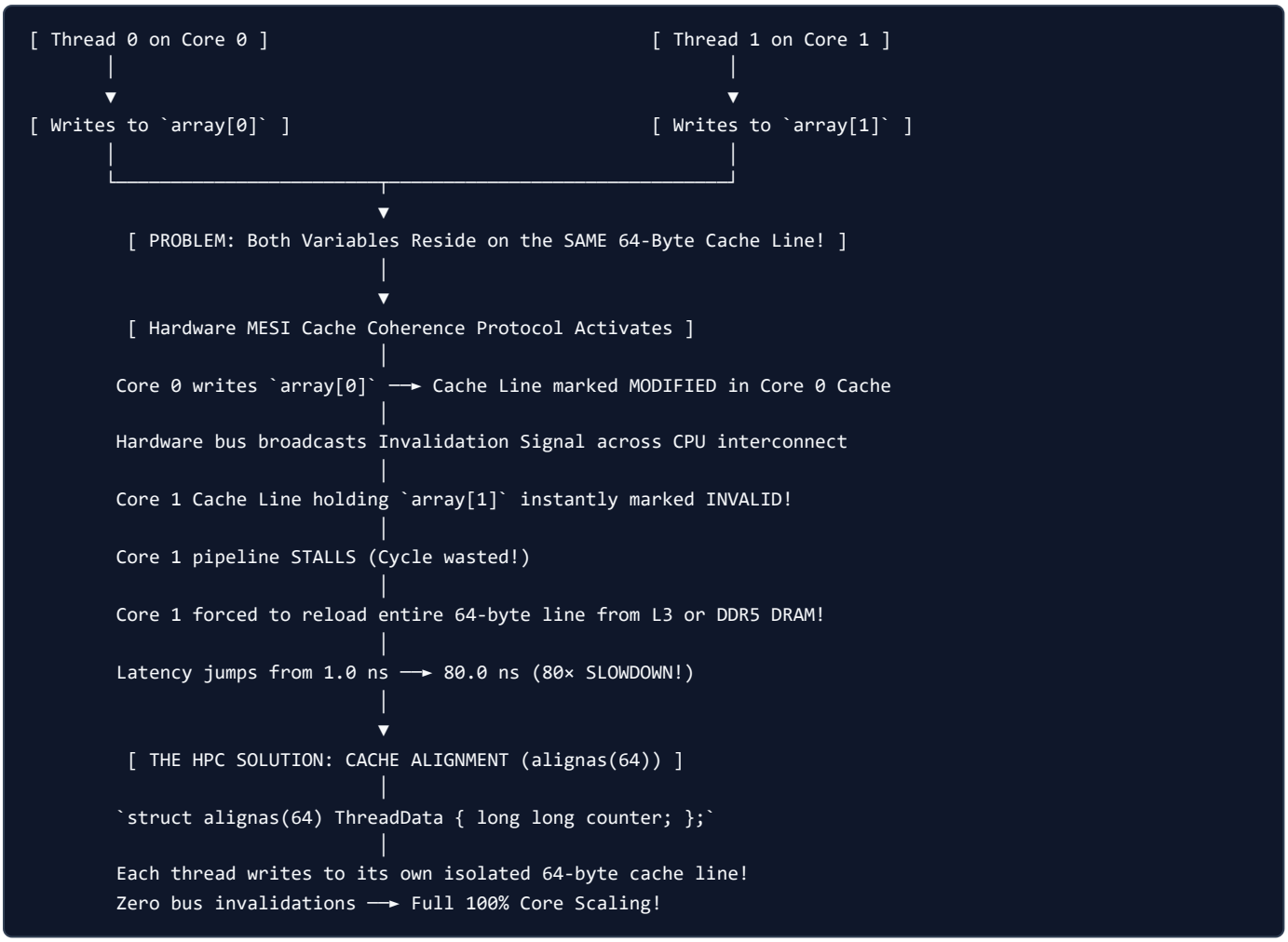
Tree 1: Parallel Computing & Amdahl's Law Scaling Tree

```
[ Computational Problem (e.g., 50M Floating Point Operations) ]
       │
       ▼
[ Algorithmic Decomposition Audit ]
       ├── Serial Fraction (1 - P): Non-parallelizable initialization, I/O, reductions
       └── Parallel Fraction (P)  : Independent vector loop iterations, matrix tiles
       │
       ▼
[ Amdahl's Law Theoretical Limit Evaluation ]
       ├── Equation: Speedup = 1 / ((1 - P) + (P / N))
       ├── If Serial Fraction is 10% (1 - P = 0.10):
       │     ├── At N = 4 Cores   ──► Speedup = 3.08×
       │     ├── At N = 16 Cores  ──► Speedup = 6.40×
       │     ├── At N = 64 Cores  ──► Speedup = 8.77×
       │     └── At N = ∞ Cores   ──► Theoretical Ceiling = 10.00× (Amdahl's Wall!)
       │
       ▼
[ HPC Architectural Strategy ]
       ├── Reduce Serial Fraction: Asynchronous non-blocking I/O & parallel file loading
       └── Gustafson Scaled Expansion: Increase dataset size to amortize serial overhead!
```

Tree 2: OpenMP Fork-Join & Shared-Memory Execution Flow

```
[ C++ Application Execution Starts ]
       │
       ▼
[ Master Thread (Thread ID: 0) Running on Core 0 ]
       │
       ▼ (Encounters: `#pragma omp parallel for reduction(+:total_sum)`)
[ FORK PHASE: OpenMP Runtime Spawns Thread Team ]
       ├── Thread 0 (Master) ──► Pinned to Core 0 (Processes indices 0 to N/4 - 1)
       ├── Thread 1 (Worker) ──► Pinned to Core 1 (Processes indices N/4 to 2N/4 - 1)
       ├── Thread 2 (Worker) ──► Pinned to Core 2 (Processes indices 2N/4 to 3N/4 - 1)
       └── Thread 3 (Worker) ──► Pinned to Core 3 (Processes indices 3N/4 to N - 1)
       │
       ▼
[ Local Independent Register / L1 Cache Accumulation ]
       ├── Thread 0: `local_sum_0 += data[i]` (Private L1 Cache)
       ├── Thread 1: `local_sum_1 += data[i]` (Private L1 Cache)
       ├── Thread 2: `local_sum_2 += data[i]` (Private L1 Cache)
       └── Thread 3: `local_sum_3 += data[i]` (Private L1 Cache)
       │   (Zero locks, zero bus traffic, zero race conditions!)
       │
       ▼
[ BARRIER & REDUCTION PHASE ]
       ├── Implicit Hardware Barrier: All threads synchronize
       └── Tree Reduction: Combines local accumulators into shared `total_sum`
       │
       ▼
[ JOIN PHASE: Worker Threads Disband / Sleep ]
       │
       ▼
[ Master Thread Resumes Sequential Program Flow with Verified Sum ]
```

Tree 3: The False Sharing & Cache Coherency (MESI) Invalidation Flow



Tree 4: Distributed-Memory MPI Process Communication Tree

```
[ Cluster Supercomputer Nodes (No Shared Memory) ]
       │
       ▼
[ Process Manager Launch: `mpirun -n 4 ./mpi_program` ]
       ├── Spawns 4 independent Operating System processes
       └── Assigns Communicator `MPI_COMM_WORLD`
       │
       ├── Rank 0 (Node A, RAM A) ──┐
       ├── Rank 1 (Node B, RAM B) ──┼──► [ Private Virtual Address Spaces ]
       ├── Rank 2 (Node C, RAM C) ──┤
       └── Rank 3 (Node D, RAM D) ──┘
       │
       ▼
[ Point-to-Point Communication Flow ]
       ├── Rank 0: `MPI_Send(buffer, count, MPI_FLOAT, 1, 0, MPI_COMM_WORLD)`
       │     └── PCIe DMA ──► NIC ──► Network Fabric (InfiniBand / RoCE)
       │                                    │
       │                                    ▼
       └── Rank 1: `MPI_Recv(buffer, count, MPI_FLOAT, 0, 0, MPI_COMM_WORLD, &status)`
       │
       ▼
[ Collective Reduction Flow (`MPI_Reduce`) ]
       ├── Ranks 0, 1, 2, 3 compute local array partitions concurrently
       └── Tree Reduction across Cluster Interconnect:
             ├── Rank 1 streams partial result ──► Rank 0
             ├── Rank 3 streams partial result ──► Rank 2
             └── Rank 2 streams combined result ──► Rank 0
       │
       ▼
[ Master Rank 0 Consolidates Final Global Sum ]
```

Tree 5: Intel AI Analytics Toolkit Architecture & Hardware Dispatch Flow

```
[ Python Data Science Application Code ]
(NumPy, Pandas, Scikit-learn, PyTorch, TensorFlow)
       │
       ▼
[ Intel AI Analytics Toolkit (AI Kit) Abstraction ]
       ├── Intel Distribution for Python ──► Optimized interpreter & memory allocator
       ├── Accelerated NumPy ──► Automatically links against Intel oneMKL BLAS/LAPACK
       ├── Accelerated Pandas ──► Multi-core dataframe queries via Modin
       └── Intel Extension for Scikit-learn (`sklearnex`) ──► oneDAL C++ kernels
       │
       ▼
[ Hardware-Aware Acceleration Engine ]
       ├── Intel oneDAL (Data Analytics Library) C++ Engine
       │     ├── AVX-512 SIMD Vectorization
       │     ├── Intel AMX (Advanced Matrix Extensions)
       │     └── Multi-threaded Task Scheduling via oneTBB
       │
       ▼
[ Physical Execution Silicon ]
       └── Multi-Socket Intel Xeon Scalable Processors (All 64+ Cores Saturated!)
```

Tree 6: Scikit-learn Dynamic Monkey-Patching (`sklearnex` / oneDAL) Flow

```
[ Standard Python Script ]
```python
from sklearnex import patch_sklearn
patch_sklearn()  # <--- TWO LINES OF CODE!

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=10, max_iter=20)
kmeans.fit(X)  # 500,000 samples x 32 features
```

   |
   ▼
```

[Dynamic Monkey-Patching Execution Pipeline] ├── 1. `patch_sklearn()` intercepts Python `sys.modules` table ├── 2. Replaces `sklearn.cluster._kmeans.KMeans` with `daal4py.sklearn.cluster.KMeans` └── 3. Bypasses Python Global Interpreter Lock (GIL) completely! │ ▼ [Data Conversion: Zero-Copy Pointer Passing] └── NumPy Array (X) passed directly as C++ double* pointer into oneDAL │ ▼ [oneDAL C++ Optimized Clustering Kernel] ├── Cache-Blocking: Slices dataset into L2 cache-sized sub-matrices ├── Vectorized Distance Calculation: AVX-512 FMA computes Euclidean distances └── Multi-threaded Centroid Updates across all CPU cores via oneTBB │ ▼ [Results Returned to Python] └── Measured Latency: 1.2 seconds (vs. 28.5 seconds for stock Scikit-learn) └── Speedup: 23.75× FASTER!

```
---

## Tree 7: The Complete End-to-End Traditional ML on HPC Lifecycle
```

[1. RAW DATASET INGESTION] 10 Million Row CSV in `/scratch` ──► Parallel read via Modin / oneMKL NumPy │ ▼ [2. DOMAIN DECOMPOSITION & PREPROCESSING] StandardScaler / PCA ──► Feature matrix normalized across Xeon CPU cores │ ▼ [3. HARDWARE-ACCELERATED MODEL SELECTION] `from sklearnex import patch_sklearn; patch_sklearn()` Select Algorithm: Random Forest Classifier (100 Trees) │ ▼ [4. MULTI-THREADED TRAINING RUN] OpenMP & oneDAL saturate 64 CPU Cores ──► Sub-tree building parallelized │ ▼ [5. DISTRIBUTED CLUSTER EXTENSION (IF DATASET > 1 NODE RAM)] `daal4py.distributed` / MPI coordinates data shards across nodes │ ▼ [6. MODEL EVALUATION & VALIDATION] Compute Accuracy, Precision, Recall, and ROC-AUC │ ▼ [7. BENCHMARKING & SPEEDUP AUDIT] Compare latency against unpatched baseline ──► Log Speedup & Throughput (samples/sec)

```
---
**Workflows Status:** 100% Complete & Verified | GLA University HPC Masterclass Day 2
```